Boolean algebra and switching functions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.1 Functions, expressions and circuits

A **switching function** is a map $f : \{0,1\}^n \to \{0,1\}$. In input order $x_1, \ldots, x_n$, there are 2^n assignments. A complete truth table specifies an output for each. Since each row has two output choices, there are 2^{2^n} switching functions.

A **Boolean expression** represents a function using AND, OR and NOT. We write $xy = x \wedge y$, $x + y = x \vee y$, and $\bar{x} = \neg x$. Here $+$ means neither integer addition nor XOR. A **literal** is a variable or its complement, a **product term** is an AND of literals, and a **sum of products**, or SOP, is an OR of product terms.

Different expressions can represent the same function, for example $AB + A\bar{B} = A$. Different circuits can also implement one function. First establish function equivalence, then design for a stated cost such as gate count, fan-in or logic depth. These costs need not equal the number of literal occurrences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.2 Minterms identify individual rows

Fix the bit order A, B, C and index $4A + 2B + C$. For an assignment $a = (a_1, \ldots, a_n)$, define

$$\ell_i(a) = \begin{cases} x_i, & a_i = 1, \\ \bar{x}_i, & a_i = 0, \end{cases} \qquad m_a = \bigwedge_{i=1}^{n} \ell_i(a).$$

This avoids a special exponent convention such as $x^0 = \bar{x}$.

At input a, every literal is 1, so $m_a = 1$. At any other input, a differing bit makes its literal 0, so $m_a = 0$. Thus m_a is true exactly on row a, and distinct minterms are mutually exclusive.

For $a = 101$, the minterm is $A\bar{B}C = m_5$; for 011, it is $\bar{A}BC = m_3$. Every three-variable minterm contains three literals. Omitting a variable accepts two rows and no longer gives a minterm.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.3 Existence and uniqueness of canonical DNF

For a fully specified function,

$$f = \bigvee_{a : f(a) = 1} m_a.$$

At any input b, all terms with $a \neq b$ vanish. If $f(b) = 1$, the right side contains $m_b = 1$; otherwise it does not. This proves the identity row by row.

Uniqueness applies to **canonical DNF**, in which every term is a complete minterm, with order and repetitions ignored. Only m_b can be true at b, so $f(b)$ uniquely determines whether that term appears. General DNF is neither unique nor necessarily minimal.

For $f = \Sigma m(3, 5, 7)$:

ABC	**000**	**001**	**010**	**011**	**100**	**101**	**110**	**111**
f	0	0	0	1	0	1	0	1

15.3 Existence and uniqueness of canonical DNF (continued)

Thus $f = \bar{A}BC + A\bar{B}C + ABC$, which simplifies to $BC + AC$. We define the empty OR as 0 and the empty AND as 1, covering constant functions and $n = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.4 Maxterms and canonical CNF

For assignment a, choose literals that are false on that row:

$$q_i(a) = \begin{cases} \bar{x}_i, & a_i = 1, \\ x_i, & a_i = 0, \end{cases} \qquad M_a = \bigvee_i q_i(a).$$

All are 0 at a; at another row at least one is 1. Thus M_a is false exactly at a. The canonical CNF identity is

$$f = \bigwedge_{a : f(a)=0} M_a.$$

At a false input b, the right side includes the zero factor M_b. At a true input, every factor is 1. This proves the identity.

For XOR, the false rows are $00, 11$, so

$$A \oplus B = (A + B)(\bar{A} + \bar{B}) = A\bar{B} + \bar{A}B.$$

DNF lists true rows; CNF lists false rows. Their lengths can differ substantially for a given function.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.5 Axioms of an abstract Boolean algebra

A **Boolean algebra** is a bounded distributive lattice in which every element has a complement. On a nonempty set B, operations $\vee, \wedge$ and constants $0, 1$ satisfy the following identities for all $x, y, z \in B$.

Property	Join form	Meet form
Commutativity	$x \vee y = y \vee x$	$x \wedge y = y \wedge x$
Associativity	$(x \vee y) \vee z = x \vee (y \vee z)$	$(x \wedge y) \wedge z = x \wedge (y \wedge z)$
Absorption	$x \vee (x \wedge y) = x$	$x \wedge (x \vee y) = x$
Bounds and identities	$x \vee 0 = x,\ x \vee 1 = 1$	$x \wedge 1 = x,\ x \wedge 0 = 0$
Distributivity	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

Each x has at least one complement y, meaning $x \vee y = 1$ and $x \wedge y = 0$. We will prove uniqueness and denote the complement by $\bar{x}$. The one-element algebra with $0 = 1$ is allowed. A nonzero algebra will explicitly require $0 \neq 1$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.6 The partial order induced by the operations

Define $x \leq y \iff x \wedge y = x$. Substituting $y = 0$ in absorption gives $x \wedge x = x$; the dual gives $x \vee x = x$. Thus the relation is reflexive.

If $x \leq y$ and $y \leq x$, then $x = x \wedge y = y \wedge x = y$, proving antisymmetry. If $x \leq y \leq z$, then

$$x \wedge z = (x \wedge y) \wedge z = x \wedge (y \wedge z) = x \wedge y = x,$$

proving transitivity. Absorption also gives

$$x \leq y \iff x \vee y = y.$$

For the forward direction, $x \vee y = (x \wedge y) \vee y = y$. For the reverse direction, $x \wedge y = x \wedge (x \vee y) = x$.

The element $x \wedge y$ is a common lower bound. If $z \leq x, y$, then $z \wedge (x \wedge y) = (z \wedge x) \wedge y = z$, so it is the greatest lower bound. Dually, $x \vee y$ is the least upper bound. Finite joins therefore have a precise least-upper-bound meaning.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.7 Unique complements, De Morgan and duality

Suppose b, c are complements of a. Distributivity gives

$$b = b \wedge 1 = b \wedge (a \vee c) = (b \wedge a) \vee (b \wedge c) = b \wedge c.$$

Similarly $c = b \wedge c$, hence $b = c$. Since a is also a complement of $\bar{a}$, uniqueness gives $\bar{\bar{a}} = a$. Also $\bar{0} = 1, \bar{1} = 0$.

Put $d = \bar{x} \vee \bar{y}$. Distributing gives

$$(x \wedge y) \wedge d = 0, \qquad (x \wedge y) \vee d = (x \vee d) \wedge (y \vee d) = 1.$$

Uniqueness of complements therefore gives

$$\overline{x \wedge y} = \bar{x} \vee \bar{y}.$$

Dually, $\overline{x \vee y} = \bar{x} \wedge \bar{y}$.

The **duality principle** interchanges $\wedge$ with $\vee$, and 0 with 1. The axiom set remains unchanged, so interchanging these symbols throughout a derivation produces another valid derivation. This operation is different from taking the pointwise complement of a function.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.8 Simplification needs a derivation and a cost model

For example,

$$\begin{aligned}
(A+B)(A+\bar{B})(\bar{A}+B) &= (A+B\bar{B})(\bar{A}+B) && \text{distributivity} \\
&= A(\bar{A}+B) && \text{complement and identity} \\
&= A\bar{A}+AB && \text{distributivity} \\
&= AB. && \text{complement and identity}
\end{aligned}$$

Every line agrees on all inputs, rather than on just a few tested rows.

If $E=(A+B)C$, its dual is $AB+C$, while its complement is $\bar{E}=\bar{A}\bar{B}+\bar{C}$. At $A=B=C=1$, the dual equals 1 and the complement equals 0, disproving their equality.

These laws also hold in abstract Boolean algebras. A table of all 2^n rows directly verifies a two-element switching function only. A theorem about abstract structures still needs a derivation from axioms or a valid representation theorem.

15.9 Implicants, primes and minimal SOP

First consider a fully specified function. Exclude identically zero terms containing both x and $\bar{x}$. A product term P is an **implicant** if $P = 1 \implies f = 1$. Deleting a literal expands its accepted assignments. An implicant is **prime** if no literal can be deleted while keeping it legal.

For $f = \Sigma m(3, 5, 7)$, ABC is an implicant but not prime. Deleting A gives BC, and deleting B gives AC; both avoid false rows. Deleting either remaining literal from AC reaches a false row.

Unless stated otherwise, a **minimal SOP** minimizes product-term count first, then literal count, in lexicographic order. A question asking only for minimal literal count uses that cost instead. Any legal term can be expanded to a prime in finitely many deletions without increasing either cost. Thus either cost has an optimum using primes only.

The optimum need not be unique. Greedily taking the largest available group need not produce it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.10 Prime charts and essential coverage

For $f = \Sigma m(3, 5, 7)$, the only maximal legal cubes are BC, AC.

Prime	$m_3 = 011$	$m_5 = 101$	$m_7 = 111$	Literals
BC	✓		✓	2
AC		✓	✓	2

Only BC covers m_3, and only AC covers m_5, making both **essential prime implicants**. Every SOP can be expanded to a prime cover, so both terms are needed. The expression

$$f = AC + BC$$

achieves two terms and four literals, proving minimality.

Factoring gives $(A + B)C$, with three literal occurrences, but that expression is not SOP. Both the expression family and the cost must be fixed before claiming a minimum.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.11 Gray code and a four-cell K-map group

A three-variable Karnaugh map can use rows $A = 0, 1$ and columns $BC = 00, 01, 11, 10$. Gray code makes neighboring columns differ in one bit, including the leftmost and rightmost columns. For

$$f(A, B, C) = \Sigma m(0, 1, 2, 3) = \bar{A},$$

the four cells lie in row $A = 0$. Both B, C vary freely in the group, leaving only $\bar{A}$.

Three-variable K-map: Σm(0,1,2,3) = ¬A

BC	00	01	11	10
A=0	000 · 1	001 · 1	011 · 1	010 · 1
A=1	100 · 0	101 · 0	111 · 0	110 · 0

A=0 stays fixed

B and C each take both values

Both are eliminated

f = ¬A

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.11 Gray code and a four-cell K-map group (continued)

The algebra matches the group exactly:

$$\bar{A}(\bar{B}\bar{C} + \bar{B}C + B\bar{C} + BC) = \bar{A}(\bar{B} + B)(\bar{C} + C) = \bar{A}.$$

No cells from a different row are included in this group.

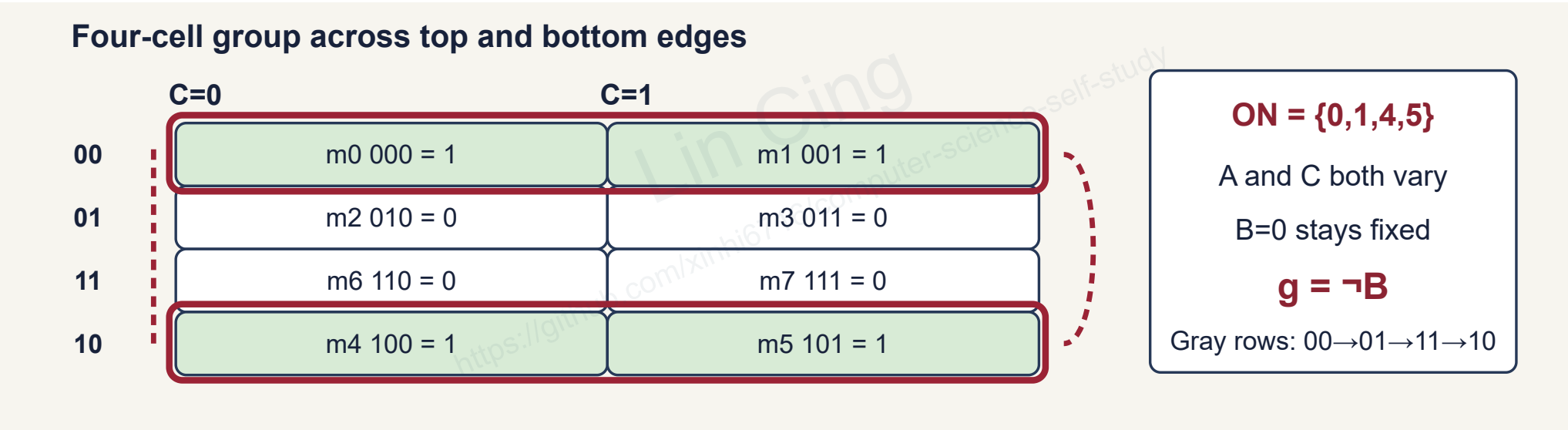
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.12 Wrap-around with a different axis layout

Now use rows $AB = 00, 01, 11, 10$ and columns $C = 0, 1$. For

$$g = \Sigma m(0, 1, 4, 5),$$

m_0, m_1 occupy the top row and m_4, m_5 the bottom row. Opposite top and bottom edges are adjacent because $AB = 00, 10$ differ only in A.



15.12 Wrap-around with a different axis layout (continued)

Within these four cells, A, C vary freely while $B = 0$ is fixed, so $g = \bar{B}$. Wrap-around draws an existing binary-cube adjacency across a planar boundary; it does not join arbitrary nonadjacent inputs.

A legal group is a cube formed by fixing some bits and allowing **all** combinations of the others. It has 2^k cells and eliminates k literals. On a three-variable map, legal groups are rectangles of $1, 2, 4, 8$ cells, allowing wrap-around and overlap. A power-of-two cell count alone is insufficient. An L-shaped incomplete cube is illegal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.13 Minimality without essential primes

Let $h = \Sigma m(1, 2, 3, 4, 5, 6)$, whose only false inputs are $000, 111$. Its six primes are:

Prime	ON indices covered	Prime	ON indices covered
$\bar{A}B$	2,3	$A\bar{B}$	4,5
$\bar{A}C$	1,3	$A\bar{C}$	4,6
$B\bar{C}$	2,6	$\bar{B}C$	1,5

Each ON row is covered by exactly two primes, so none is essential. Every single literal includes 000 or 111, making it illegal. Every legal term covers at most two ON rows, so at least three terms, each with at least two literals, are necessary.

Both covers attain three terms and six literals:

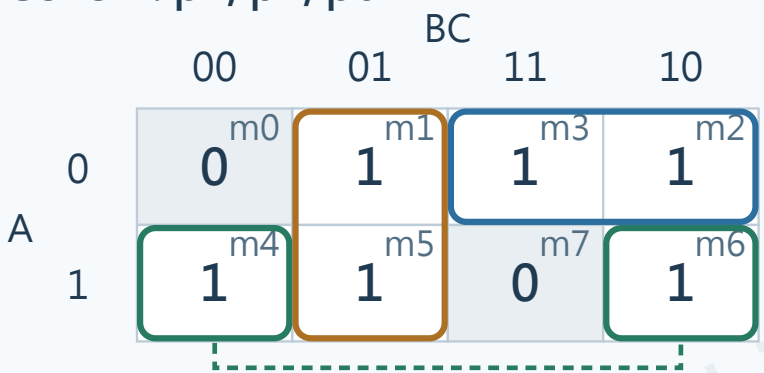
$$h = \bar{A}B + A\bar{C} + \bar{B}C = \bar{A}C + A\bar{B} + B\bar{C}.$$

These are distinct optimal SOPs. Finding all essential primes does not necessarily finish minimization; a remaining chart still requires a cover choice.

Two optimal covers on the same K-map

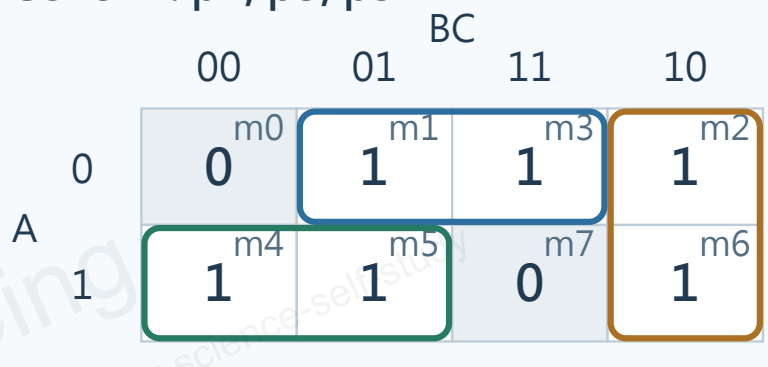
Gray columns: BC=00,01,11,10. Green end cells form one wrap-around pair.

Cover I: p2, p4, p6



p2: (¬A)B {2,3}
p4: A(¬C) {4,6}
p6: (¬B)C {1,5}

Cover II: p1, p5, p3



p1: (¬A)C {1,3}
p5: A(¬B) {4,5}
p3: B(¬C) {2,6}

Each cover uses three two-literal terms and covers all six 1-cells. Exercise 7 proves both optimality and completeness.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.14 A general cube algorithm

Represent cubes using $0, 1, -$, with $1{-}1$ denoting AC. Two cubes can merge only when their dash positions agree and exactly one remaining fixed bit differs, as in $101, 111 \to 1{-}1$. The merged cube is their union. Cubes with arbitrary mismatched dash positions cannot be combined.

Start with all legal minterms and merge level by level. Mark terms contained in larger cubes. The legal cubes that cannot expand are primes. To prove completeness, induct on the number of free bits. A legal cube with k free bits splits into two legal subcubes by fixing one free bit to 0 and 1. Both halves are generated at the preceding level and merge back to the cube.

Build a prime chart with required ON rows as columns and primes as rows. Select rows covering all columns under the term-count and literal-count objective. Exhaustive search, branch and bound, or exact cover search can solve this selection. Sorting primes by coverage size is only a heuristic and has no general optimality guarantee.

As the variable count grows, the truth table, prime set and cover search may all become large. Convenient small K-maps do not imply a simple polynomial-time minimizer for arbitrary instances.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.15 Don't-care specifications

Let the ON set F, OFF set R and don't-care set D be pairwise disjoint and partition all assignments. A legal implementation g must equal 1 on F and 0 on R. Choosing either output on each row of D gives a **completion**.

A legal cube for a partial specification need only avoid R. Generate primes using $F \cup D$, but require coverage only of F; a prime covering only don't-care rows need not be selected. This differs from $P \implies f$ for a fully specified function, since no output has yet been assigned to f on D.

For $F = \{0, 2, 4\}, D = \{1, 5\}, R = \{3, 6, 7\}$, one implementation is

$$g = \bar{B} + \bar{A}\bar{C}.$$

The first term accepts $0, 1, 4, 5$; the second accepts $0, 2$. Their union is $0, 1, 2, 4, 5$, which includes all ON rows and no OFF row, and chooses 1 for both don't-care rows.

If later requirements fix outputs on inputs 1 or 5, the specification has changed. The chosen completion must then be checked against those new requirements.

15.16 A literal-count lower bound

For the same specification $F = \{0, 2, 4\}, D = \{1, 5\}, R = \{3, 6, 7\}$, now minimize total literal occurrences.

Among the six single literals, only $\bar{B}$ avoids the OFF set, and it misses ON row $2 = 010$. The other literals $A, \bar{A}, B, C, \bar{C}$ hit OFF rows $6, 3, 3, 3, 6$, respectively.

An SOP using at most two literal occurrences cannot work:

- Two one-literal terms can only repeat $\bar{B}$, still missing 2.
- A single two-literal term cannot cover $0, 2, 4$. Their only common fixed bit is $C = 0$, so there is no second fixed bit. Even the larger cube $\bar{C}$ hits OFF row 6.
- Constant 1 hits the OFF set, while constant 0 misses the ON set.

At least three literals are therefore necessary. The expression $\bar{B} + \bar{A}\bar{C}$ attains this bound. Legality and minimality require different arguments. A truth-table check alone establishes legality.

15.17 Shannon expansion and universal gates

Fix a variable x, and let $f_0 = f|_{x=0}$, $f_1 = f|_{x=1}$. Then

$$f = \bar{x} f_0 + x f_1.$$

At $x = 0$, the right side is f_0; at $x = 1$, it is f_1. These two cases prove the identity on every input. This is also the selection rule of a 2-to-1 multiplexer.

Define NAND by $x \uparrow y = \overline{xy}$. Two-input NAND gates alone give

$$\bar{x} = x \uparrow x, \qquad xy = (x \uparrow y) \uparrow (x \uparrow y), \qquad x + y = (x \uparrow x) \uparrow (y \uparrow y).$$

NAND can implement AND, OR and NOT, and canonical DNF represents every switching function using those operations. Thus NAND is **functionally complete**. The dual argument proves the same for NOR.

This proves that a circuit exists. It does not minimize gate count or model physical loading and propagation delay.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.18 A circuit with four NAND gates

For $f = A\bar{B} + BC$, define

$$n_B = \mathrm{NAND}(B, B) = \bar{B}, \quad t_1 = \mathrm{NAND}(A, n_B) = \overline{A\bar{B}}, \quad t_2 = \mathrm{NAND}(B, C) = \overline{BC}.$$

Finally,

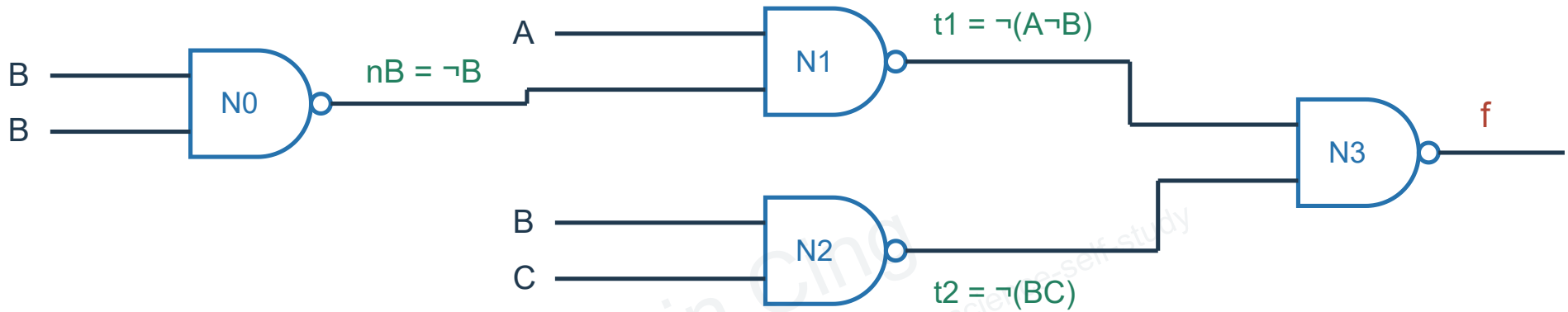
$$f = \mathrm{NAND}(t_1, t_2) = \overline{\overline{A\bar{B}}\,\overline{BC}},$$

which is the required OR by De Morgan's law.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.18 A circuit with four NAND gates (continued)

f = A¬B + BC wired with four two-input NAND gates



Output bubbles invert. Both N0 inputs receive B. Following the wires gives f = ¬(t1·t2).

Both input pins of N0 receive B, so N0 is an inverter. If $\bar{B}$ is externally available, omit N0. The diagram has four two-input NAND gates and maximum logic depth three, along B through N0, N1 and N3. Depth counts gate levels; physical time also depends on each gate's delay model.

15.19 Verify intermediate signals along the wires

Evaluate every wire of the preceding circuit, including each output inversion bubble.

ABC	n_B	t_1	t_2	**NAND output**	$A\bar{B} + BC$
000	1	1	1	0	0
001	1	1	1	0	0
010	0	1	1	0	0
011	0	1	0	1	1
100	1	0	1	1	1
101	1	0	1	1	1
110	0	1	1	0	0
111	0	1	0	1	1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.19 Verify intermediate signals along the wires (continued)

The last two columns agree on every row, verifying the function. Four identical gates do not mean four gate delays in series; N1 and N2 lie on parallel branches. A static truth table also makes no guarantee about fan-out, wire delay or transient hazards.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.20 Sets and divisibility as abstract examples

The power set $\mathcal{P}(S)$, with union as join, intersection as meet and complement relative to S, satisfies the axioms. Its zero is $\varnothing$, and its one is S.

Another example uses the positive divisors of 30:

$$B = \{1, 2, 3, 5, 6, 10, 15, 30\}, \qquad x \wedge y = \gcd(x, y), \quad x \vee y = \operatorname{lcm}(x, y).$$

Associate each divisor with the subset of prime factors it contains, for example $6 \leftrightarrow \{2, 3\}$ and $10 \leftrightarrow \{2, 5\}$. Since $30 = 2 \cdot 3 \cdot 5$ has no repeated prime factor, gcd and lcm become intersection and union. The complement is $30/x$.

Here zero is the numeral 1, and one is the numeral 30, not the identities of ordinary integer arithmetic. For example,

$$6 \wedge 10 = 2, \qquad 6 \vee 10 = 30, \qquad \bar{6} = 5.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

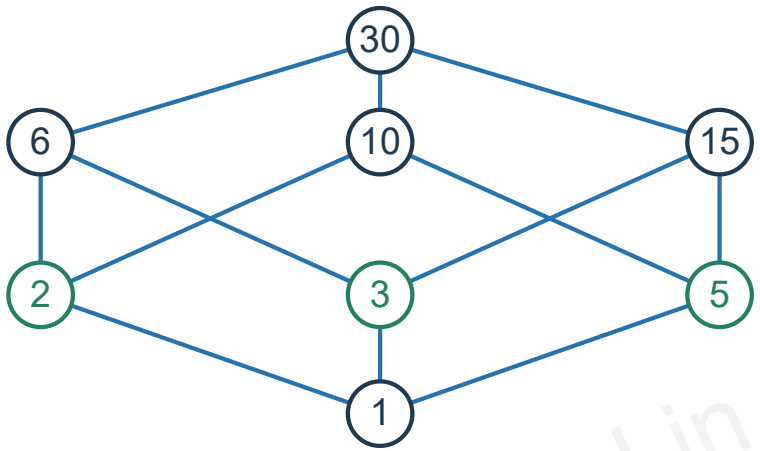
15.21 Atoms in a Hasse diagram

An **atom** is a minimal nonzero element: $a \neq 0$, and $0 < y \leq a \implies y = a$. The atoms of the divisor algebra of 30 are $2, 3, 5$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.21 Atoms in a Hasse diagram (continued)

Divisors of 30: divisibility corresponds to subsets of three atoms



Zero = 1; one = 30 (divisibility order)

Atoms: 2 ↔ {2}, 3 ↔ {3}, 5 ↔ {5}

meet = gcd; join = lcm

6 ∧ 10 = 2; 6 ∨ 10 = 30

¬6 = 5; {2,3} has complement {5}

Every edge is a cover directed upward by divisibility. Only circles are elements; crossings are not vertices.

Moving upward along a cover adds one previously absent prime factor. Not every comparable pair is joined by an edge. The eight subsets of the three atoms correspond exactly to the eight divisors.

Not every divisor lattice is Boolean. A complement b of 2 among divisors of 12 would require $\gcd(2, b) = 1$ and $\operatorname{lcm}(2, b) = 12$. Only $1, 3$ are coprime to 2, and their lcms with 2 are $2, 6$. Thus 2 has no complement.

15.22 Nonzero elements of a finite algebra have atoms below them

Let B be a finite Boolean algebra and $x \neq 0$. The set $\{y : 0 < y \leq x\}$ is nonempty because it contains x. A finite partial order has a minimal element a in this set. If $0 < z \leq a$, transitivity gives $z \leq x$, and minimality forces $z = a$. Hence a is an atom.

Distinct atoms a, b satisfy $a \wedge b = 0$. Otherwise $0 < a \wedge b \leq a, b$, so the atom property forces $a \wedge b = a = b$, a contradiction.

This does not make every nonzero element an atom. The divisor 6 is the join of two atoms and is not an atom. Finiteness matters: in an infinite structure, a descending chain may have no minimal nonzero element, invalidating the selection argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.23 Each element is the join of its atoms

Put $A_x = \{a : a \text{ is an atom and } a \leq x\}$, and let $s = \bigvee A_x$, with empty join 0. Since each $a \leq x$, we have $s \leq x$.

If $s \neq x$, then $x \wedge \bar{s} \neq 0$. Otherwise,

$$x = x \wedge (s \vee \bar{s}) = (x \wedge s) \vee (x \wedge \bar{s}) = s,$$

a contradiction. By the preceding result, an atom a lies below $x \wedge \bar{s}$. Since $a \leq x$, it belongs to A_x, so $a \leq s$. Also $a \leq \bar{s}$, forcing $a \leq s \wedge \bar{s} = 0$, another contradiction.

Therefore

$$x = \bigvee_{a \leq x,\ a \text{ an atom}} a.$$

In particular, all atoms join to 1. This argument is the core of the finite representation theorem; a few Hasse diagrams do not establish the general statement.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.24 Atoms and joins

For an atom a, distributivity gives

$$a \wedge (x \vee y) = (a \wedge x) \vee (a \wedge y).$$

Each meet on the right is either 0 or a. Therefore

$$a \leq x \vee y \iff a \leq x \text{ or } a \leq y.$$

The reverse direction follows from the upper-bound property. For the forward direction, if both meets were zero, the displayed expression would contradict $a \neq 0$.

Similarly, $a \leq x \wedge y$ exactly when both $a \leq x$ and $a \leq y$. For complements,

$$a = (a \wedge x) \vee (a \wedge \bar{x}).$$

At least one term is a, and both cannot equal a. Thus

$$a \leq \bar{x} \iff a \not\leq x.$$

The atom property and distributivity are essential here. The implication $z \leq x \vee y \implies z \leq x \text{ or } z \leq y$ is false for arbitrary elements.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.25 Representation of finite Boolean algebras

Let A be the set of all atoms. Define $\Phi : B \to \mathcal{P}(A)$ by $\Phi(x) = \{a \in A : a \leq x\}$. The preceding results give

$$\Phi(x \wedge y) = \Phi(x) \cap \Phi(y), \quad \Phi(x \vee y) = \Phi(x) \cup \Phi(y), \quad \Phi(\bar{x}) = A \setminus \Phi(x).$$

Also $\Phi(0) = \varnothing$, $\Phi(1) = A$, so every operation is preserved.

If $\Phi(x) = \Phi(y)$, the atom-join theorem gives $x = y$, proving injectivity. For any $T \subseteq A$, set $x = \bigvee T$. Each atom of T lies in $\Phi(x)$. If another atom $b \leq x$, repeatedly applying the join property gives $b \leq a$ for some $a \in T$, forcing $b = a$. The empty case has $x = 0$ and also works. Thus $\Phi(x) = T$, proving surjectivity.

Consequently $B \cong \mathcal{P}(A)$. With k atoms, $|B| = 2^k$. This is an isomorphism of operations, not merely an equality of cardinalities. The degenerate algebra has $k = 0$ and corresponds to $\mathcal{P}(\varnothing) = \{\varnothing\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.26 An infinite algebra without atoms

The finite theorem used an atom below every nonzero element. For an infinite counterexample, take all finite unions of half-open arcs with dyadic endpoints on a circle of circumference 1, together with the empty set. Complements, intersections and unions remain of this form, so this is a Boolean algebra.

Every nonempty element contains a half-open interval of length 2^{-r}. Bisecting it gives a strictly smaller nonempty element in the algebra. Thus there are no atoms. Yet the empty set and the whole circle differ, so the algebra is nonzero.

Without finiteness, neither “no atoms implies a one-element algebra” nor “every Boolean algebra is the power set of its atoms” follows. General representation theorems for infinite Boolean algebras use different objects; the finite proof above does not establish them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.27 Boolean algebras and Boolean rings

For sets, symmetric difference corresponds to XOR. In a Boolean algebra, define

$$x \oplus y = (x \wedge \bar{y}) \vee (\bar{x} \wedge y), \qquad xy = x \wedge y.$$

For finite algebras, Φ transforms these into symmetric difference and intersection. They form a commutative ring whose zero and identity are the Boolean $0, 1$, and $x^2 = x$.

Conversely, suppose a ring with identity satisfies $x^2 = x$ for every element. Then $(x + x)^2 = x + x$ gives $4x = 2x$, hence $2x = 0$. Expanding $(x + y)^2 = x + y$ gives $xy + yx = 0$. Since every z satisfies $2z = 0$, this implies $xy = yx$. This includes the degenerate zero ring. Define

$$x \wedge y = xy, \qquad \bar{x} = 1 + x, \qquad x \vee y = x + y + xy.$$

The two uses of addition must be distinguished. Ring $+$ is XOR, while Boolean join is OR. At $x = y = 1$, their results are 0 and 1, respectively, in a nonzero algebra.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.28 Verifying the Boolean-ring construction

We know the ring is commutative, $2x = 0$ and $x^2 = x$. Define $x \le y \iff xy = x$. Idempotence proves reflexivity, and commutativity proves antisymmetry. If $xy = x, yz = y$, then $xz = xyz = xy = x$, proving transitivity. Also $0 \le x \le 1$.

The product xy is the greatest lower bound: $(xy)x = xy$, and $zx = z, zy = z$ imply $zxy = z$. Put $j = x + y + xy$. Since $xj = x, yj = y$, it is a common upper bound. If $xz = x, yz = y$, then $jz = x + y + xy = j$, so it is the least upper bound. Uniqueness of greatest lower bounds and least upper bounds gives commutativity, associativity and absorption for these lattice operations.

The complement $1 + x$ satisfies $x(1 + x) = 0$ and $x \vee (1 + x) = 1$. Both distributive laws follow by calculation:

$$x(y \vee z) = xy + xz + xyz = (xy) \vee (xz),$$

$$(x \vee y)(x \vee z) = x + yz + xyz = x \vee (yz).$$

The expansions use $x^2 = x$ and $2t = 0$. Thus the construction satisfies the complete Boolean-algebra axioms, rather than merely resembling their notation.

15.29 What to reconstruct independently

- Fix the bit order, derive each minterm and maxterm from its row, and prove the canonical forms equivalent row by row.
- Distinguish legal implicants, primes, essential primes and optimal covers. Give a lower bound and a construction under a fixed expression family and cost.
- Read K-map axes and Gray code, identify the fixed bits of each group, and check that no OFF row is covered.
- Derive all intermediate NAND signals along the actual wires. A static truth table does not establish delay or hazard behavior.
- Derive the order, complements and atoms from the axioms, and prove the finite representation theorem rather than only memorizing 2^k.

The exercises and solutions develop each requirement. Their cost models and finiteness hypotheses are part of the questions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study